

SKILL SESSION 9&10

Session 9 :

TASK 1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

TASK 2 : To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
[(base) saiabburi@Charishma-MacBook-Air ~ % mkdir SkillWeek5
[(base) saiabburi@Charishma-MacBook-Air ~ % cd SkillWeek5
[(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % nano session9.c
```

```
[(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % clang session9.c -o session9
[(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % ls
session9      session9.c
[(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % ./session9
Simple Shell - Session 9
Type 'help' to see commands.
shell> pwd
/Users/saiabburi/SkillWeek5
shell> cd..
Invalid command: cd..
shell> cd
Directory changed successfully.
shell> pwd
/Users/saiabburi
shell> cd -
Directory changed successfully.
shell> pwd
/Users/saiabburi/SkillWeek5
shell> cd abc123
cd: No such file or directory
shell> echo Hello Charishma
Hello Charishma
shell> help

Built-in commands:
pwd    - display current directory
cd     - change directory
cd -   - return to previous directory
echo   - display text
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#define MAX_INPUT 1024

char previous_dir[PATH_MAX] = "";

void pwd_command() {
    char current_dir[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) != NULL) {
        printf("%s\n", current_dir);
    } else {
        perror("pwd");
    }
}

void cd_command(char *path) {
    char current_dir[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        return;
    }

    if (path == NULL || strlen(path) == 0) {
        path = getenv("HOME");

        if (path == NULL) {
            printf("HOME is not set\n");
            return;
        }
    }

    if (strcmp(path, "-") == 0) {
        if (strlen(previous_dir) == 0) {
            printf("No previous directory\n");
            return;
        }

        path = previous_dir;
    }
}
```

```

    if (chdir(path) == -1) {
        perror("cd");
        return;
    }

    strcpy(previous_dir, current_dir);

    printf("Directory changed successfully.\n");
}

void echo_command(char *text) {
    if (text != NULL) {
        printf("%s\n", text);
    }
}

void help_command() {
    printf("\nBuilt-in commands:\n");
    printf("pwd    - display current directory\n");
    printf("cd      - change directory\n");
    printf("cd -    - return to previous directory\n");
    printf("echo    - display text\n");
    printf("help    - display available commands\n");
    printf("exit    - exit shell\n");
}

void exit_command() {
    printf("Exiting shell...\n");
    exit(0);
}

typedef void (*CommandFunction)(char *);

typedef struct {
    char *name;
    CommandFunction function;
} Command;

void pwd_wrapper(char *arg) {
    pwd_command();
}

```

```
void help_wrapper(char *arg) {
    help_command();
}

void exit_wrapper(char *arg) {
    exit_command();
}

Command command_table[] = {
    {"pwd", pwd_wrapper},
    {"cd", cd_command},
    {"echo", echo_command},
    {"help", help_wrapper},
    {"exit", exit_wrapper}
};

int command_count = sizeof(command_table) / sizeof(command_table[0]);

int main() {
    char input[MAX_INPUT];

    printf("Simple Shell - Session 9\n");
    printf("Type 'help' to see commands.\n");

    while (1) {
        printf("shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0) {
            continue;
        }
    }
}
```

```

char *command = strtok(input, " ");
char *argument = strtok(NULL, "");

int found = 0;

for (int i = 0; i < command_count; i++) {
    if (strcmp(command, command_table[i].name) == 0) {
        found = 1;

        if (strcmp(command, "cd") == 0) {
            cd_command(argument);
        } else if (strcmp(command, "echo") == 0) {
            echo_command(argument);
        } else {
            command_table[i].function(argument);
        }

        break;
    }
}

if (!found) {
    printf("Invalid command: %s\n", command);
}

return 0;
}

```

Session 10 :

TASK 1 : To Retrieve Current Directory, Display Path,
Process Exit Requests, Cleanup Resources, Save State,
Verify Correct Behavior

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <ctype.h>

#define MAX_INPUT 1024

char previous_dir[PATH_MAX] = "";

void save_state() {
    FILE *file = fopen("shell_state.txt", "w");

    if (file == NULL) {
        perror("state file");
        return;
    }

    char current_dir[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) != NULL) {
        fprintf(file, "Last Directory: %s\n", current_dir);
    }

    fclose(file);
}

void pwd_command() {
    char current_dir[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) != NULL) {
        printf("%s\n", current_dir);
    } else {
        perror("getcwd");
    }
}

void cd_command(char *path) {
    char current_dir[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        return;
    }
}
```

```

    if (path == NULL || strlen(path) == 0) {
        path = getenv("HOME");

        if (path == NULL) {
            printf("HOME is not set\n");
            return;
        }
    }

    if (strcmp(path, "-") == 0) {
        if (strlen(previous_dir) == 0) {
            printf("No previous directory\n");
            return;
        }

        path = previous_dir;
    }

    if (chdir(path) == -1) {
        perror("cd");
        return;
    }

    strcpy(previous_dir, current_dir);

    printf("Directory changed successfully.\n");
}

int valid_variable_name(char *name) {
    if (name == NULL || strlen(name) == 0) {
        return 0;
    }

    if (!isalpha(name[0]) && name[0] != '_') {
        return 0;
    }

    for (int i = 1; name[i] != '\0'; i++) {
        if (!isalnum(name[i]) && name[i] != '_') {
            return 0;
        }
    }

    return 1;
}

```

```

void export_command(char *argument) {
    if (argument == NULL) {
        printf("Usage: export NAME=VALUE\n");
        return;
    }

    char *equals = strchr(argument, '=');

    if (equals == NULL) {
        printf("Invalid export syntax. Use NAME=VALUE\n");
        return;
    }

    *equals = '\0';

    char *name = argument;
    char *value = equals + 1;

    if (!valid_variable_name(name)) {
        printf("Invalid variable name: %s\n", name);
        return;
    }

    if (setenv(name, value, 1) != 0) {
        perror("export");
        return;
    }

    printf("Exported %s=%s\n", name, value);
}

void echo_command(char *argument) {
    if (argument == NULL) {
        printf("\n");
        return;
    }
}

```

```

    if (argument[0] == '$') {
        char *value = getenv(argument + 1);

        if (value != NULL) {
            printf("%s\n", value);
        } else {
            printf("\n");
        }
    } else {
        printf("%s\n", argument);
    }
}

void help_command() {
    printf("\nAvailable commands:\n");
    printf("pwd                - display current directory\n");
    printf("cd PATH              - change directory\n");
    printf("cd -                  - previous directory\n");
    printf("echo TEXT            - display text\n");
    printf("echo $NAME            - display environment variable\n");
    printf("export NAME=VALUE    - set environment variable\n");
    printf("help                  - display help\n");
    printf("exit                  - save state and exit\n");
}

void exit_command() {
    printf("Saving shell state...\n");

    save_state();

    printf("Cleaning up resources...\n");
    printf("Exiting shell...\n");

    exit(0);
}

typedef void (*CommandFunction)(char *);

typedef struct {
    char *name;
    CommandFunction function;
} Command;

```

```

void pwd_wrapper(char *arg) {
    pwd_command();
}

void cd_wrapper(char *arg) {
    cd_command(arg);
}

void echo_wrapper(char *arg) {
    echo_command(arg);
}

void export_wrapper(char *arg) {
    export_command(arg);
}

void help_wrapper(char *arg) {
    help_command();
}

void exit_wrapper(char *arg) {
    exit_command();
}

Command command_table[] = {
    {"pwd", pwd_wrapper},
    {"cd", cd_wrapper},
    {"echo", echo_wrapper},
    {"export", export_wrapper},
    {"help", help_wrapper},
    {"exit", exit_wrapper}
};

int command_count = sizeof(command_table) / sizeof(command_table[0]);

int main() {
    char input[MAX_INPUT];

    printf("Simple Shell - Session 10\n");
    printf("Type 'help' to see commands.\n");

    while (1) {
        printf("shell10> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0) {
            continue;
        }

        char *command = strtok(input, " ");
        char *argument = strtok(NULL, "");

        int found = 0;

        for (int i = 0; i < command_count; i++) {
            if (strcmp(command, command_table[i].name) == 0) {
                found = 1;
                command_table[i].function(argument);
                break;
            }
        }

        if (!found) {
            printf("Invalid command: %s\n", command);
        }

        save_state();

        return 0;
    }
}

```

```

(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % nano session10.c
(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % clang session10.c -o session10
(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % LS
session10      session10.c      session9      session9.c
(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % ls
session10      session10.c      session9      session9.c
(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % ./session10
Simple Shell - Session 10
Type 'help' to see commands.
shell10> pwd
/Users/saiabburi/SkillWeek5
shell10> cd
Directory changed successfully.
shell10> pwd
/Users/saiabburi
shell10> cd -
Directory changed successfully.
shell10> pwd
/Users/saiabburi/SkillWeek5

```

TASK 2 : To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```

shell10> export NAME=Charishma
Exported NAME=Charishma
shell10> echo $NAME
Charishma
shell10> export COLLEGE=Engineering
Exported COLLEGE=Engineering
shell10> echo $COLLEGE
Engineering
shell10> export NAME=Student
Exported NAME=Student
shell10> echo $NAME
Student
shell10> export 123NAME=hello
Invalid variable name: 123NAME
shell10> export NAME-TEST=hello
Invalid variable name: NAME-TEST
shell10> export HELLO
Invalid export syntax. Use NAME=VALUE
shell10> v
Invalid command: v
shell10> export HELLO=WORLD
Exported HELLO=WORLD
shell10> echo $HELLO
WORLD

```

```
shell10> help
```

```
Available commands:
```

```
pwd          - display current directory
cd PATH      - change directory
cd -         - previous directory
echo TEXT    - display text
echo $NAME   - display environment variable
export NAME=VALUE - set environment variable
help        - display help
exit        - save state and exit
```

```
shell10> abcd
```

```
Invalid command: abcd
```

```
shell10> exit
```

```
Saving shell state...
```

```
Cleaning up resources...
```

```
Exiting shell...
```

```
[(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % ls
```

```
session10      session10.c    session9      session9.c    shell_state.txt
```

```
[(base) saiabburi@Charishma-MacBook-Air SkillWeek5 % cat shell_state.txt
```

```
Last Directory: /Users/saiabburi/SkillWeek5
```

```
(base) saiabburi@Charishma-MacBook-Air SkillWeek5 %
```